

COSC2129/1476

Artificial Intelligence



Adversarial Search

Lecturer: Thuy Nguyen

Office location: 2.4.27

Road Map for Today

- ❑ Revision of classic search strategies
- ❑ Games as adversarial search
- ❑ Minimax search
- ❑ How to improve the efficiency of minimax search?
 - Alpha-beta pruning
 - Cutting off search
 - Heuristic evaluation functions
- ❑ How to deal with games with chances?

Book AIMA, Chapter 6

▼ Chapter 6: Adversarial Search and Games

6.1 Game Theory

6.2 Optimal Decisions in Games

6.3 Heuristic Alpha–Beta Tree Search

6.4 Monte Carlo Tree Search

6.5 Stochastic Games

6.6 Partially Observable Games

6.7 Limitations of Game Search Algorithms

Summary

Classic Search Strategies

Uninformed (blind) search

- ❑ Breadth-first search (BFS)
- ❑ Uniform-cost search (UCS)
- ❑ Depth-first search (DFS)
- ❑ Depth-limited search (DLS)
- ❑ Iterative deepening search (IDS)
- ❑ Bi-directional search (BDS)



Informed (heuristic) search

- ❑ Greedy search
- ❑ A* search

Performance of Search Strategies

Completeness: Is it guaranteed to find a solution if one exists?

Optimality: Will it find an optimal solution?

Time complexity: How many nodes get expanded?

Space complexity: How much memory is needed?

Time and space complexity is calculated based on:

b	Branching factor
d	Depth of the shallowest goal node
m	Maximum length of any path in the state space

Properties of Search Strategies

- ❑ **Breadth-first (BFS):** FIFO Q, complete (if b is finite), optimal (if step cost is identical)
- ❑ **Uniform-cost (UCS):** Priority Q ordered by $g(n)$, complete (if b is limited and step cost is positive), optimal
- ❑ **Depth-first (DFS):** LIFO Q, non-complete, non-optimal
- ❑ **Depth-limited (DLS):** LIFO Q, non-complete, non-optimal
- ❑ **Iterative deepening (IDS):** LIFO Q, complete (if b is finite), optimal (if step cost is identical)
- ❑ **Bi-directional (BDS):** complete (if b is finite and both directions are BFS), optimal (if step cost is identical and both directions are BFS)
- ❑ **Greedy:** Priority Q ordered by $h(n)$, complete (if graph search in finite spaces), non-optimal
- ❑ **A*:** Priority Q ordered by $g(n) + h(n)$, complete (if b is finite and step cost is positive), optimal (if $h(n)$ is admissible for tree search or consistent for graph search)

Properties of Heuristic Functions

Admissibility

$h(n)$ **admissible** if it never overestimates the cost from n to reach the goal.

This means that A^* search approaches the optimal cost C^* from below.

Every node with $f(n) < C^*$ will be expanded.

Consistency

$h(n) \leq c(n, a, n') + h(n')$, n' is any successor of n

Informedness

$h_2(n)$ is **more informed** than $h_1(n)$ if $h_2(n) \geq h_1(n)$.

Why is $h_2(n)$ better than $h_1(n)$?

Heuristics allow us to prune the search space: if $h_2(n) \geq h_1(n)$ then $h_2(n)$ will make us to search only a subset of the nodes searched by $h_1(n)$.

Games



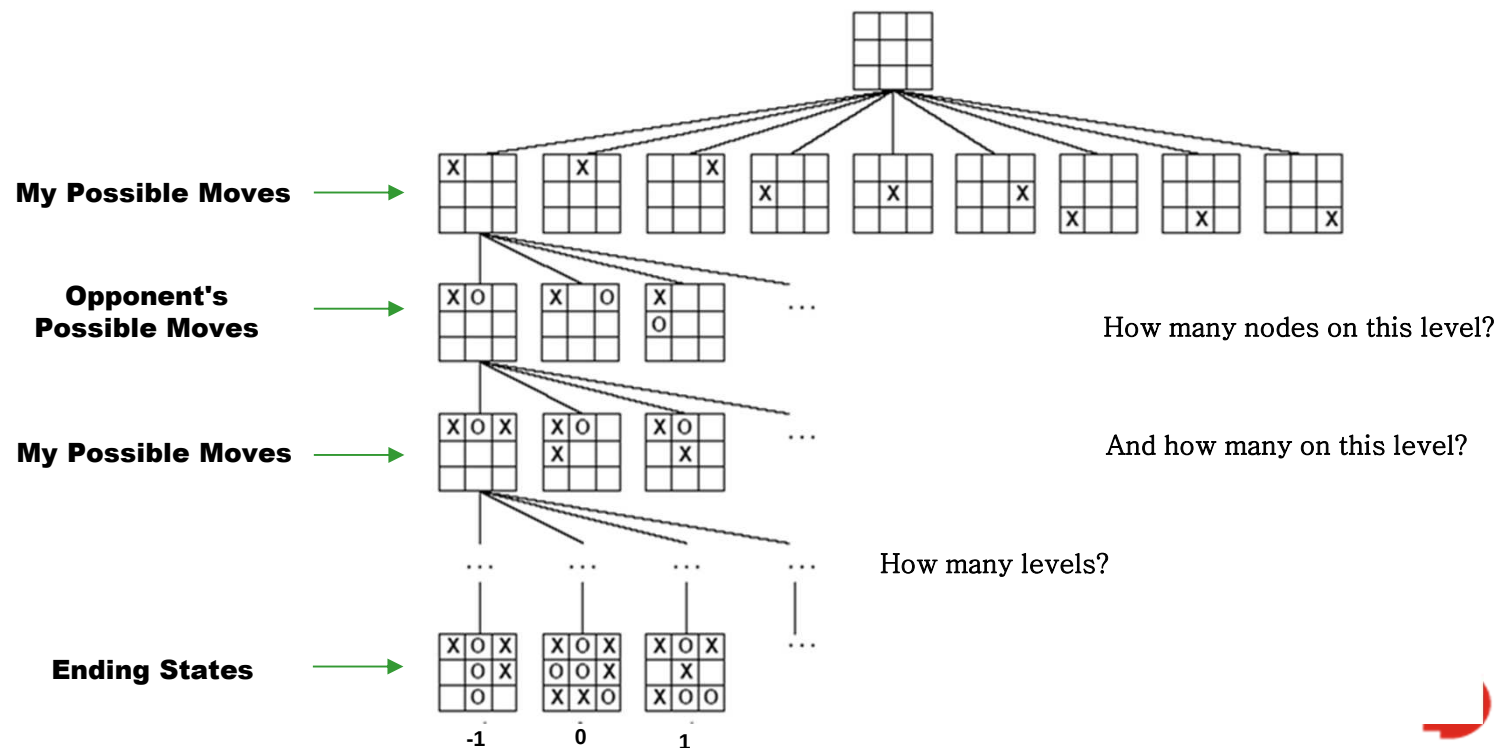
Games as Adversarial Search

8-puzzle problems involve only one player. A game usually have two or more players, such as chess, tic-tac-toe, checker, Othello, etc.

Can we formulate game playing as search problems? **YES**

Adversarial search if the goals of different players are **conflict**!

Can we build a search tree to represent a game from the very beginning to every possible ending scenario of the game? **YES**, such as the tree for tic-tac-toe:



Games as Adversarial Search (cont'd)

- It is almost **impossible (and also unnecessary)** to build a search tree to represent the entire search space for even a simple game like tic-tac-toe.
- A realistic search method is to find the “best” next move based on the current state of the game. This move should ideally maximize the chance of winning.
- To find this move, we need to search through the search space (entire?), starting from the current state.
- The opponent will reply **against** our move. Then we perform the search **again** based on the **new state** to find the next “best” move.
- This process will repeat until the game is over.
- In game playing, **minimax search** is often used to find next moves.



Definitions

A game can be formally defined as a kind of search problem with the following elements:

- ❑ Initial state s_0 : How is the game set up at the start
- ❑ Player(s): Who will move in state s
- ❑ Actions(s): The set of legal moves in state s
- ❑ Result(s, a): transition model defining the result of a move
- ❑ Terminal-test(s): true if the game is over (i.e., s is a terminal state)
- ❑ Utility(s, p): utility value for a game ending at a terminal state s for a player p
e.g., utility value for chess is: -1, 0 or 1 (zero-sum game)



Minimax Game

Two players: **MAX** and **MIN**

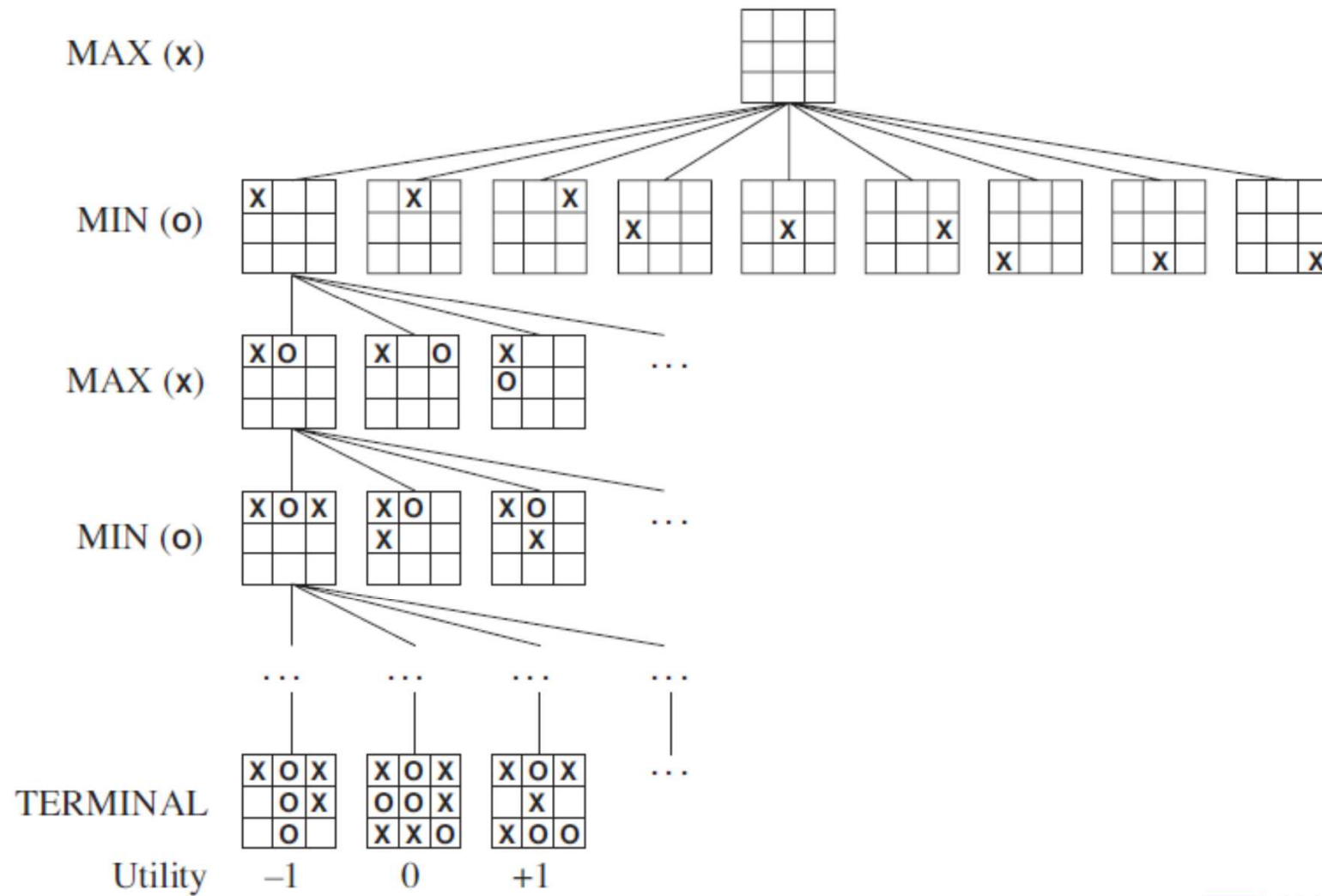
Initial state is a **MAX** node

- Children of **MAX** nodes are **MIN** (or leaf) nodes
- Children of **MIN** nodes are **MAX** (or leaf) nodes

Calculate **Minimax(s)**

- **Utility(s)** if **Terminal-test(s)**
- $\text{Max}_a \text{Minimax}(\text{Result}(s, a))$ if $\text{Player}(s) = \text{MAX}$
- $\text{Min}_a \text{Minimax}(\text{Result}(s, a))$ if $\text{Player}(s) = \text{MIN}$
- Calculate in the **depth-first** manner

Minimax Game Example



Ply

Depth limit of minimax games can be specified via **ply**:

2-ply: 1 move for me, 1 move for my opponent

4-ply: 2 moves each

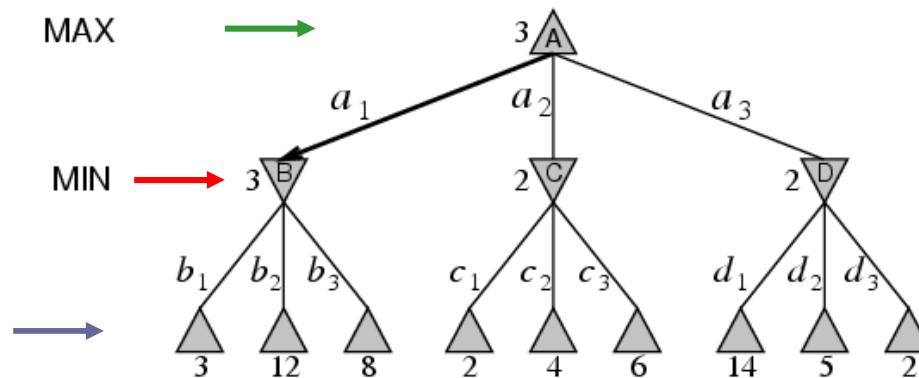
...

2n-ply: n moves each

Puts a depth bound on minimax search

Minimax Search Algorithm

Consider a trivial game that only involves two moves from start to end:



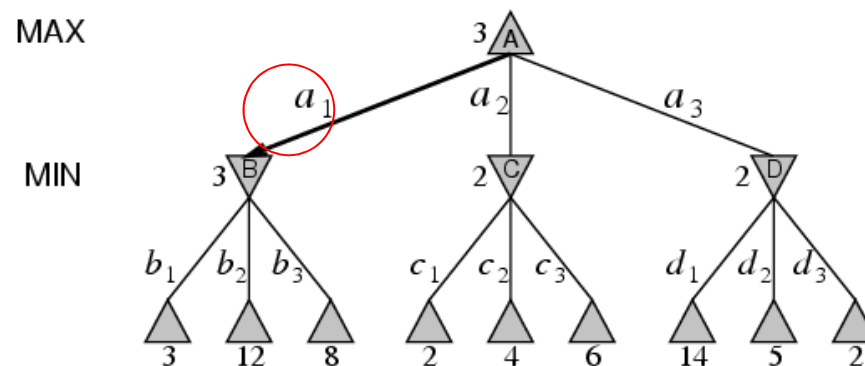
- ▲ is a "MAX node". It is MAX's turn to move (to find the max value).
- ▼ is a "MIN node". It is MIN's turn to move (to find the min value).

The bottom nodes are leave nodes on which utility function applies.

The sequence of assigning values to nodes is:

$3 \rightarrow b_1, 12 \rightarrow b_2, 8 \rightarrow b_3, 3 \rightarrow B, 2 \rightarrow c_1, 4 \rightarrow c_2, 6 \rightarrow c_3, 2 \rightarrow C, 14 \rightarrow d_1, 5 \rightarrow d_2, 2 \rightarrow d_3, 2 \rightarrow D, 3 \rightarrow A$

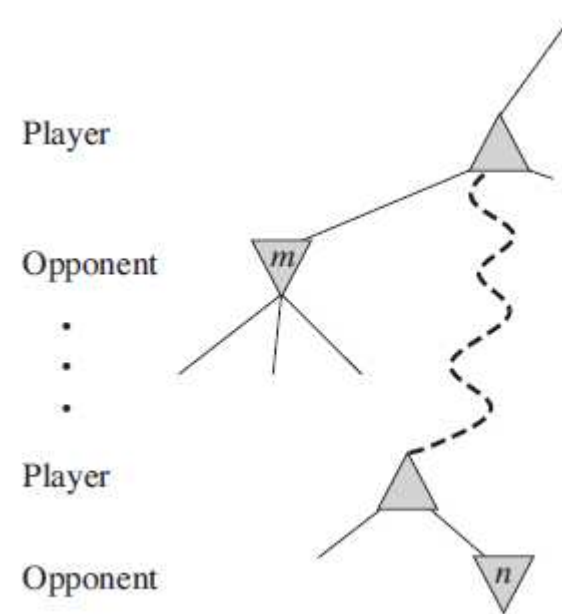
Minimax Search Algorithm (cont'd)



In this example, MAX's best move at the root node is a_1 because it leads to the successor with the highest minimax value. MIN's best reply is b_1 because it leads to the successor with the lowest utility value.

Always assume that the opponent will play perfectly (infallible).

Alpha-Beta Pruning



Alpha-Beta Pruning

Not all branches need expanding.

If one **MAX's** child has value **3**, no **MIN's** child (with **this MAX** being along its path to the root) having value **3 or less** will change this.

If one **MIN's** child has value **2**, no **MAX's** child (with **this MIN** being along its path to the root) having value **2 or more** will change this.

The highest current value of any **MAX** node along the path to the root is **alpha**.

The lowest current value of any **MIN** node along the path to the root is **beta**.

Alpha-Beta Pruning (cont'd)

Same search procedure as for minimax except:

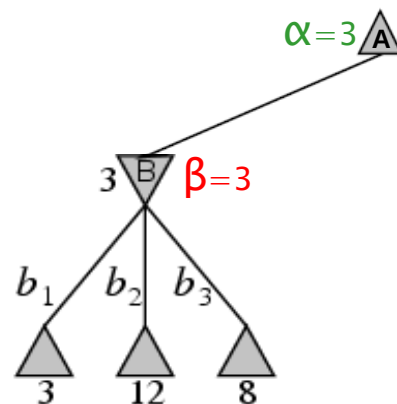
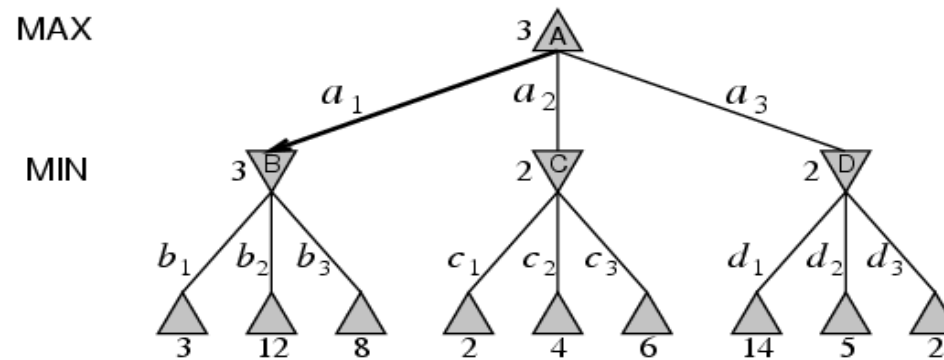
MIN node: stop expansion if current value $\leq$ alpha of its parent (this node can't be higher than alpha)

MAX node: stop if current value $\geq$ beta of its parent (this node can't be lower than beta)

This also means that the whole subtree for a pruned node is not expanded.

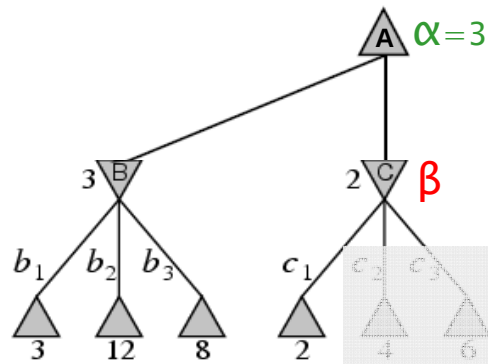
Alpha-Beta Pruning (cont'd)

We use the same two-ply game tree as an example for α - β pruning.



- Expand the first branch
- Get utility values for each terminal node.
- So B's current utility value (β) is 3.
- So A's current utility value (α) is 3.

Alpha-Beta Pruning (cont'd)



- Expand node C by c_1
- Reach the terminal node
- The utility value of this terminal node $< \alpha=3$.
- So C 's current utility value is 2, and no further expansion on C .

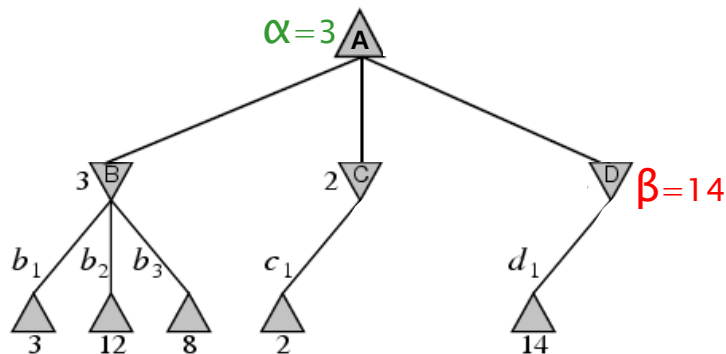
A beta-pruning is performed here - a MIN cannot have a higher value than its parent MAX.

What if we expand by c_2 and c_3 ? The corresponding utility values are 4, 6. So β of C will remain 2. Minimax search will still not choose C (because at least B has a higher value 3).

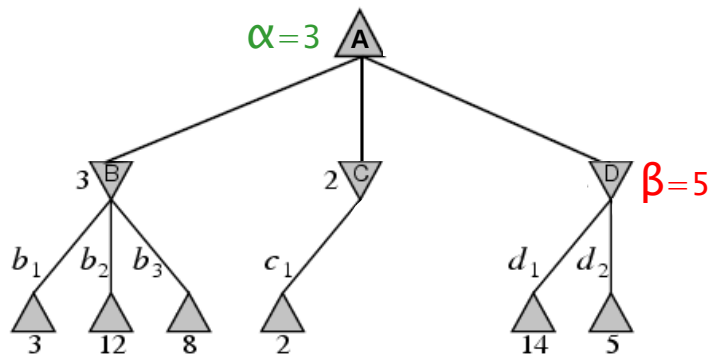
The search outcome is not affected by the utility values of the terminal nodes led by c_2 and c_3 .

So expanding by c_2 and c_3 are unnecessary.

Alpha-Beta Pruning (cont'd)



- Expand node D by d_1
- Reach the terminal node
- The utility value of this terminal node $> \alpha=3$.
- So D's current utility value is 14, and **continue** the expansion on D.

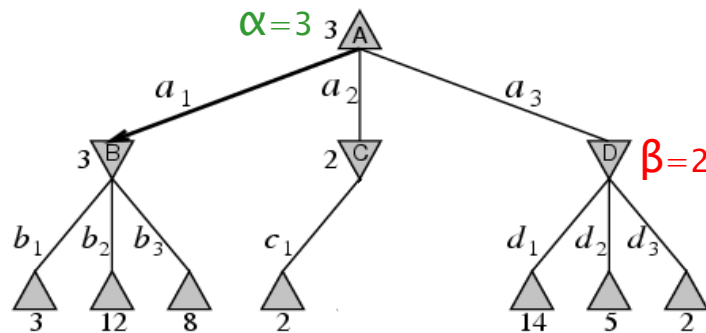


- Expand node D by d_2
- Reach the terminal node
- The utility value of this terminal node $> \alpha=3$.
- So D's current utility value is 5, and **continue** the expansion on D.

Can we stop expansion at d_2 ?

No! What if $d_3 = 2, 4, 10$?

Alpha-Beta Pruning (cont'd)



- Expand node D by d_3 .
- Reach the terminal node.
- The utility value of this terminal node $< \alpha=3$, and the final branch
- So D's current utility value is 2, and no further expansion on D.
- Identify a_1 as the next move.

The outcome of this alpha-beta pruning is same with that from minimax. Less nodes are expanded here. So alpha-beta pruning is more efficient.

However, the expansion of d_1 or d_2 seems not worthwhile. The low value of d_3 bring down the value of D to 2.

Alpha-Beta Pruning (cont'd)

- ❑ Quality of pruning depends on the order of expansion
- ❑ Try to generate children in “desired” order
- ❑ Can reduce minimax from $O(b^m)$ to $O(b^{m/2})$
- ❑ Reduces effective branching factor from b to $b^{1/2}$
- ❑ Searches a tree twice as deep as minimax in the same amount of time

Search for Games

- ❑ Set up a game as a kind of search tree
- ❑ Use minimax method to choose moves
- ❑ Use alpha-beta pruning to cut down alternatives
- ❑ Some implementation tricks: transposition tables as hash tables.

Search for Games (cont'd)

- ❑ Games involve search
- ❑ Search space can be huge (e.g., chess: 1040 states)
- ❑ Not all moves may be chosen by the opponent(s)
- ❑ Time limit may degrade search
- ❑ Exhaustive search is not how humans do it!

Search for Games (cont'd)

- ❑ Assume optimal opponent
- ❑ Prune search space
- ❑ Use heuristic approximation of the expected utility
- ❑ Deal with chance and imperfect information

Heuristic Evaluation Function

Idea goes back to Claude Shannon in 1950(!)

Limit depth and estimate utility via a heuristic evaluation function (educated guess)

Calculate $H\text{-Minimax}(s)$

- $\text{Eval}(s)$ if $\text{Cutoff-test}(s, d)$
- $\text{Max}_a H\text{-Minimax}(\text{Result}(s, a), d+1)$ if $\text{Player}(s) = \text{MAX}$
- $\text{Min}_a H\text{-Minimax}(\text{Result}(s, a), d+1)$ if $\text{Player}(s) = \text{MIN}$

Heuristic Evaluation Function (cont'd)

So what is a good evaluation function?

- ❑ Computationally efficient
- ❑ Match the ranking order of the utility values of terminal states
e.g., winning better than draws better than losses
- ❑ Strongly correlate with actual winning chances on non-terminal states
- ❑ Will have some level of uncertainty (as it is an *estimate*)

Heuristic Evaluation Function (cont'd)

Often a **weighted sum** of various features:

$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

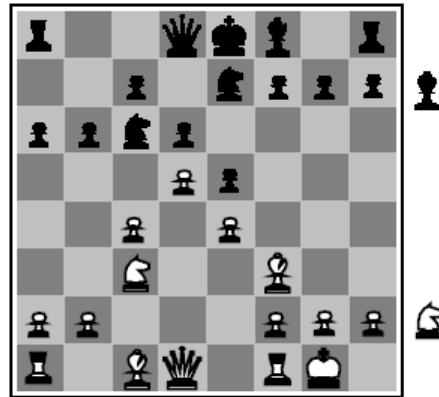
Assumes features are **independent**.

Example chess function:

$$9 * \# \text{Queens} + 4 * \# \text{Bishops} + 3 * \# \text{Knights} + 1 * \# \text{pawns}$$

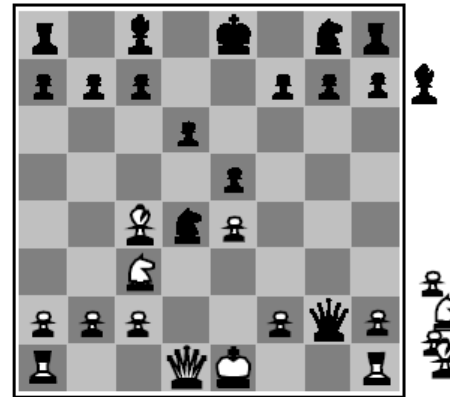
Features and weights are NOT part of rules of chess, but come from previous experience.

Heuristic Evaluation Function (cont'd)



Black to move

White slightly better



White to move

Black winning

For chess the weights and factors could be:

$$w_1 f_1(s) = 9 * (\text{number of white Queen} - \text{number of black queen})$$

$$w_2 f_2(s) = 4 * (\text{number of white Bishop} - \text{number of black Bishop})$$

$$w_3 f_3(s) = 3 * (\text{number of white knight} - \text{number of black knight})$$

$$w_4 f_4(s) = 1 * (\text{number of white pawn} - \text{number of black pawn})$$

... so on and so forth...

By this simple function we can estimate which player is closer to winning.

Heuristic Evaluation Function (cont'd)

Apply only to **quiescent** states (not 'I am about to lose my queen!')

Can check for **non-quiescent** states (i.e., something is under attack) and insist that they are expanded

horizon effect and singular extension

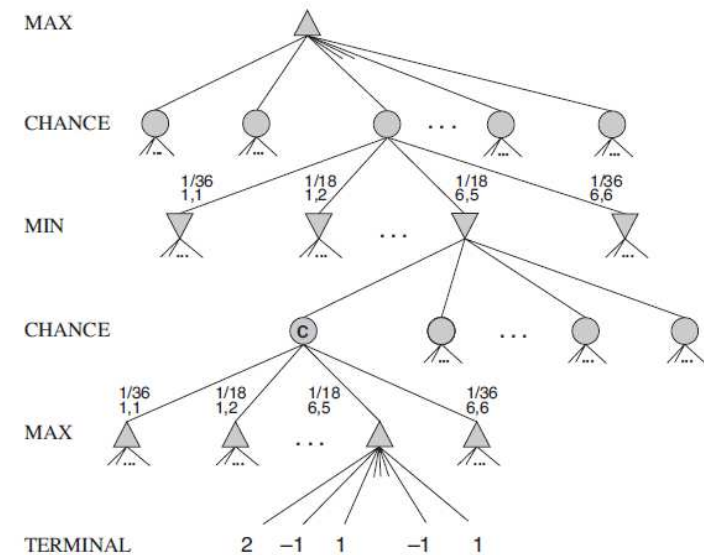
Different Types of Games

	Perfect Information	Imperfect Information
Deterministic	Chess, Draughts (checkers), Go, Othello, Noughts and Crosses (tic-tac-toe), ...	Kriegspiel, Battleship
Stochastic	Backgammon, Monopoly	Bridge, 500, Scrabble

Stochastic Games

Involving some "chance" factors:

- ❑ Dice
- ❑ Deal in a card game
- ❑ Tossing a coin



Backgammon:

- ❑ Deterministic moves, but which ones are legal are determined by two dices
- ❑ Require **chance nodes** in the game tree

Stochastic Games (cont'd)

- Calculate $\text{ExpectMinimax}(s)$ as:
 - $\text{Utility}(s)$ if $\text{Terminal-test}(s)$
 - $\text{Max}_a \text{ExpectMinimax}(\text{Result}(s,a))$ if $\text{Player}(s) = \text{MAX}$
 - $\text{Min}_a \text{ExpectMinimax}(\text{Result}(s,a))$ if $\text{Player}(s) = \text{MIN}$
 - $\text{SUM}_r P(r) * \text{ExpectMinimax}(\text{Result}(s,r))$ if $\text{Player}(s) = \text{CHANCE}$

- Complicate alpha-beta pruning

- Can use Monte Carlo (random) choices over a large enough sample

State-of-the-art Game Programs

❑ Chess

Deep Blue (1997)

- alpha-beta search on lots of specialist processors
- Typically 14-ply, sometimes up to 40-ply (!!)
- Opening book of 4000 positions
- Database of 700,000 grandmaster games
- Endgame database of all 5 piece checkmates
- A good PC with the right program can match a human world champion ...

❑ Draughts: Chinook, using alpha-beta search, beat human master in 1990; Now plays perfectly with a vast endgame database

❑ Othello: Computers too good for humans

❑ Backgammon: Computers competitive with humans

❑ Go: Humans still ahead (branch factor 361 ...); Monte Carlo methods used

Conclusions

- ❑ Revision of classic search strategies
- ❑ Games as adversarial search
- ❑ Minimax search: assuming optimal players
- ❑ How to improve the efficiency of minimax search?
 - Alpha-beta pruning
 - Cutting off search
 - Heuristic evaluation functions
- ❑ How to deal with games with chance?

Questions?
